Universitat Politècnica de Catalunya (UPC)

Asignatura: Microprocesadores y Sistemas Digitales · Curso 2025-2026

PRÁCTICA 3C

Bluetooth Low Energy (BLE) con ESP32

Usando Inteligencia Artificial como herramienta de desarrollo

Alumno: Bernat

Dispositivo: ESP32-S3 · Nombre BLE: ESP32-S3_BERNAT

Repositorio: https://github.com/bf-upc/Practica3C

Fecha: 18 de marzo de 2026

1. Introducción al Protocolo BLE

Bluetooth Low Energy (BLE) es una variante del estándar Bluetooth diseñada para aplicaciones con restricciones de consumo energético. A diferencia del Bluetooth Clásico (SPP), BLE está optimizado para transmitir pequeñas cantidades de datos de forma periódica consumiendo energía mínima, lo que lo hace ideal para sensores, wearables y dispositivos IoT.

1.1 Modelo GATT y conceptos clave

Concepto	Descripción	En esta práctica
Peripheral	Dispositivo que ofrece servicios BLE	ESP32-S3 como servidor
Central	Se conecta y consume los servicios	Móvil con nRF Connect
Service	Agrupación lógica de características	UUID: 4fafc201...
Characteristic	Dato individual: READ/WRITE/NOTIFY	ADC, LED, JSON, AUTH
Descriptor BLE2902	CCCD: activa/desactiva notificaciones	Valor 0x0001 = activo
Notify	El servidor empuja datos sin polling	ADC cada 1 segundo

1.2 MTU y limitaciones de frecuencia de notificación

El MTU (Maximum Transmission Unit) en BLE define el tamaño máximo de un paquete de datos. Por defecto es de 23 bytes (20 bytes útiles), aunque puede negociarse hasta 512 bytes con BLE 4.2+. En esta práctica los datos enviados son pequeños (int ADC = 4 bytes, JSON ~40 bytes), por lo que el MTU por defecto es suficiente y no se requiere negociación.

Respecto a la frecuencia máxima de notificaciones, BLE no puede enviar notificaciones ilimitadas: el intervalo de conexión mínimo es de 7.5 ms (configurable con setMinPreferred), lo que da un techo teórico de ~133 notificaciones/segundo. En la práctica, con un intervalo de conexión de 30 ms (el que negocia nRF Connect, visible en el log), el máximo real es ~33 notificaciones/segundo. El valor elegido de 1 notificación/segundo está muy por debajo de este límite, garantizando fiabilidad total.

1.3 BLE vs Bluetooth Clásico en ESP32

Característica	BLE	Bluetooth Clásico (SPP)
Consumo energético	Muy bajo	Alto

Coexistencia con WiFi	Buena (time-sharing radio 2.4GHz)	Problemática
Velocidad de datos	Baja (ideal sensores)	Mayor
Librería Arduino	BLEDevice.h, BLEServer.h	BluetoothSerial.h
Soporte en ESP32-S3	Sí (nativo)	No disponible

2. Cuestionario de Comprensión BLE

Antes de comenzar el desarrollo, se realizó el cuestionario de 8 preguntas indicado en el enunciado para verificar la comprensión de los conceptos BLE.

RESULTADO DEL CUESTIONARIO BLE

8 / 8 — 10/10 ✓ Aprobado en el primer intento

Número de intentos necesarios: 1

2.1 Preguntas y respuestas

P1 (test): ¿Qué propiedad permite al servidor enviar datos sin que el cliente los solicite? → **C) NOTIFY ✓**

P2 (test): ¿Qué valor hay que escribir en el CCCD para activar notificaciones? → **B) 0x0001 ✓**

P3 (test): ¿Cómo se llama el dispositivo que ofrece servicios BLE? → **C) Peripheral ✓**

P4 (V/F): "BLE y BT Clásico coexisten sin problema con WiFi" → **Falso ✓** — El ESP32 tiene una sola radio 2.4GHz. El ESP32-S3 además no soporta BT Clásico, solo BLE.

P5 (V/F): "Un UUID de 128 bits identifica de forma única un servicio o característica BLE" → **Verdadero ✓**

P6 (abierta): READ: el cliente pide los datos cuando quiere (pull). NOTIFY: el cliente se suscribe vía CCCD y el servidor envía automáticamente (push).

P7 (abierta): El BLE2902 evita gastar energía enviando datos a nadie. Permite saber si hay un cliente suscrito, respetando la filosofía de bajo consumo de BLE.

P8 (práctica): MyCCCDCallbacks es un callback que se ejecuta cuando el cliente escribe en el descriptor 0x2902. Lee 2 bytes little-endian como uint16_t; si vale 0x0001 activa notificationsEnabled.

3. Desarrollo de la Práctica

3.1 Fase 1 — Código base con IA

Se utilizó DeepSeek como herramienta de IA para generar el código base. El primer prompt solicitó un servidor BLE completo con las características fundamentales del enunciado:

Prompt 1 enviado a DeepSeek:

"Genera un sketch completo de Arduino para ESP32 que implemente un servidor BLE con: un servicio con UUID '4fafc201-...', una característica de lectura del ADC en GPIO4, una característica de escritura para controlar el LED integrado, mensajes Serial al conectar/desconectar, y comentarios explicando cada bloque."

El código funcionó correctamente tras ajustar GPIO36 → GPIO4 (ESP32-S3) y el nombre BLE a 'ESP32-S3_BERNAT'.

3.2 Fase 2 — Notificaciones automáticas (Prompt 2)

Prompt 2:

"Modifica el código para que la característica ADC envíe una notificación automática al cliente cada 1 segundo sin polling. Usa el descriptor BLE2902."

La IA añadió MyCCCDCallbacks para detectar la activación del CCCD y el flag notificationsEnabled que controla el envío desde el loop().

3.3 Fase 3 — Característica JSON (Prompt 3)

Prompt 3:

"Añade una tercera característica de lectura que devuelva un string JSON: {\"adc\": valor, \"led\": true/false, \"uptime\": segundos_desde_arranque}"

La IA implementó updateJsonCharacteristic() que construye el JSON manualmente sin librerías externas, manteniéndolo sincronizado tanto en lecturas como al cambiar el estado del LED.

3.4 Ejercicio A — BLE + WiFi AP simultáneo (Prompt 4)

Prompt 4:

"Combina el sketch BLE con un servidor WiFi en modo Access Point. El LED debe controlarse tanto desde BLE como desde una página web. Usa una variable global compartida para el estado del LED."

La función applyLed(state, source) centraliza el control del LED para ambas interfaces, garantizando coherencia del estado independientemente del origen del comando:

```
void applyLed(bool state, const char* source) {
    ledState = state;                          // variable global compartida
    digitalWrite(LED_BUILTIN, state ? HIGH : LOW);
    updateJsonCharacteristic();                // JSON siempre sincronizado
    Serial.printf("LED %s via %s\n", state ? "ON":"OFF", source);
}
```

El ESP32-S3 expone el AP WiFi 'ESP32-S3_BERNAT' (contraseña: 12345678) y sirve la página de control en http://192.168.4.1. En el loop() se atienden tanto server.handleClient() como las notificaciones BLE, demostrando la coexistencia real de ambos protocolos sobre la radio 2.4GHz mediante time-sharing.

3.5 Ejercicio C — Seguridad BLE con PIN (Prompt 5)

> **Prompt 5:**
> *"Añade al sketch una característica de autenticación BLE con PIN de 6 dígitos. El cliente debe escribir el PIN correcto en una nueva característica AUTH antes de poder controlar el LED. Si el PIN es incorrecto, el acceso queda bloqueado."*

Se añadió una cuarta característica (UUID ...26ab, WRITE|READ) que implementa el control de acceso por PIN. El flag bleAuthenticated se resetea a false en cada nueva conexión, obligando a re-autenticar en cada sesión. El JSON incluye el campo "auth" para inspeccionar el estado de autenticación:

```
class AuthCharacteristicCallbacks : public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pCharacteristic) {
        std::string pin = pCharacteristic->getValue();
        if (pin == CORRECT_PIN) {           // CORRECT_PIN = "123456"
            bleAuthenticated = true;
            pAuthCharacteristic->setValue("OK");
        } else {
            bleAuthenticated = false;
            pAuthCharacteristic->setValue("FAIL");
        }
    }
};
```

La característica LED comprueba bleAuthenticated antes de ejecutar cualquier acción. Si el cliente no se ha autenticado, responde con "AUTH_REQUIRED" y rechaza el comando:

```
void onWrite(BLECharacteristic *pCharacteristic) {
    if (!bleAuthenticated) {
        pLedCharacteristic->setValue("AUTH_REQUIRED");
        return;   // comando rechazado
    }
    // ... control normal del LED
}
```

4. Descripción del Código Final

4.1 UUIDs — cuatro características

```
#define SERVICE_UUID             "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHARACTERISTIC_ADC_UUID  "beb5483e-36e1-4688-b7f5-ea07361b26a8"  // READ
| NOTIFY
```

```
#define CHARACTERISTIC_LED_UUID  "beb5483e-36e1-4688-b7f5-ea07361b26a9"  // READ
| WRITE (protegido)
#define CHARACTERISTIC_JSON_UUID "beb5483e-36e1-4688-b7f5-ea07361b26aa"  // READ
#define CHARACTERISTIC_AUTH_UUID "beb5483e-36e1-4688-b7f5-ea07361b26ab"  // READ
| WRITE (PIN)
```

4.2 Tabla de bloques funcionales

Bloque / Clase	Función
MyServerCallbacks	Gestiona conexión/desconexión. Resetea bleAuthenticated y reinicia advertising al desconectar.
AuthCharacteristicCallbacks	Valida el PIN escrito por el cliente. Responde 'OK' o 'FAIL' y actualiza bleAuthenticated.
LedCharacteristicCallbacks	Comprueba bleAuthenticated. Si no autenticado responde 'AUTH_REQUIRED'. Si autenticado llama applyLed().
MyCCCDCallbacks	Detecta activación (0x0001) o desactivación (0x0000) de notificaciones ADC.
JsonCharacteristicCallbacks	Actualiza JSON (con campo auth) justo antes de cada lectura del cliente.
applyLed(state, source)	Punto único de control del LED. Llamado desde BLE y web. Garantiza coherencia del estado compartido.
readADCSafe()	Lee el ADC con validación de rango (0-4095). Devuelve 0 si el valor es anómalo.
handleLed() — Web	Procesa GET /led?state=1|0, llama applyLed() y redirige a / con HTTP 303.
loop()	Atiende server.handleClient() y envía notify() BLE cada 1 segundo si hay cliente suscrito y autenticado.

4.3 Flujo de comunicación completo

```
[nRF Connect]            [ESP32-S3]          [Navegador Web]
     |                        |                      |
     |--- WRITE AUTH('123456')>|  bleAuthenticated=true
     |<-- READ AUTH = 'OK' ----|
     |--- WRITE LED('1') ----->|  applyLed(true, BLE)
     |<-- BLE NOTIFY(ADC) -----|  cada 1 segundo
     |                        |<-- GET /led?state=1--|  applyLed(true,
Web)
     |--- READ JSON ---------->|  {adc,led,uptime,auth:true}
  [sin auth] WRITE LED('1') -->|  → 'AUTH_REQUIRED'
```

5. Resultados — Servidor BLE (Fases 1-3)

5.1 Monitor serie — Inicialización

Al cargar el sketch, el monitor serie confirma la inicialización correcta de todas las interfaces:

```
🔌 ================================
🔌 Servidor BLE ESP32-S3
🔌 Notificaciones automáticas + JSON
🔌 ================================
📋 Modelo: ESP32-S3
📋 LED_BUILTIN: GPIO97
💡 LED configurado y apagado
📊 ADC en GPIO4
📡 BLE inicializado
📊 Característica ADC: READ | NOTIFY
   └─ Descriptor BLE2902 configurado
💡 Característica LED: WRITE
📋 Característica JSON: READ
   └─ Formato: {"adc":123, "led":true/false, "uptime":1234}
✅ Servicio iniciado
----------------------------------------
📱 Busca 'ESP32-S3_BERNAT' en nRF Connect
💡 Envía '1' o '0' para controlar LED
📋 Característica JSON (READ) - Lee estado completo:
   {"adc":valor, "led":true/false, "uptime":segundos}
🔔 Para notificaciones ADC: escribe 0100 en descriptor CCCD
========================================
```

Fig. 1 — Monitor serie: inicialización del servidor BLE ESP32-S3

5.2 Monitor serie — Conexión y notificaciones ADC

Ciclo completo de conexión: cliente BLE conectado, activación del CCCD (valor 0x1) y notificaciones automáticas ADC con voltaje calculado cada segundo:

```
✅ Cliente BLE conectado
⏸️ Conectado - Esperando activación de notificaciones...
   En nRF Connect: busca el descriptor CCCD y escribe 0100
⏸️ Conectado - Esperando activación de notificaciones...
   En nRF Connect: busca el descriptor CCCD y escribe 0100
⏸️ Conectado - Esperando activación de notificaciones...
   En nRF Connect: busca el descriptor CCCD y escribe 0100
⏸️ Conectado - Esperando activación de notificaciones...
   En nRF Connect: busca el descriptor CCCD y escribe 0100
⏸️ Conectado - Esperando activación de notificaciones...
   En nRF Connect: busca el descriptor CCCD y escribe 0100
⏸️ Conectado - Esperando activación de notificaciones...
   En nRF Connect: busca el descriptor CCCD y escribe 0100
📖 Cliente leyendo característica JSON
📋 JSON actualizado: {"adc":121,"led":false,"uptime":34}
🔔 Notificaciones ADC ACTIVADAS por el cliente
📋 CCCD valor: 0x1
🔔 NOTIFICACIÓN - ADC: 127 (0.10V) | LED: OFF
🔔 NOTIFICACIÓN - ADC: 139 (0.11V) | LED: OFF
🔔 NOTIFICACIÓN - ADC: 139 (0.11V) | LED: OFF
🔔 NOTIFICACIÓN - ADC: 148 (0.12V) | LED: OFF
🔔 NOTIFICACIÓN - ADC: 123 (0.10V) | LED: OFF
🔔 NOTIFICACIÓN - ADC: 136 (0.11V) | LED: OFF
📖 Cliente leyendo característica JSON
🔔 NOTIFICACIÓN - ADC: 99 (0.08V) | LED: OFF
📖 Cliente leyendo característica JSON
📋 JSON actualizado: {"adc":121,"led":false,"uptime":44}
🔔 NOTIFICACIÓN - ADC: 111 (0.09V) | LED: OFF
🔔 NOTIFICACIÓN - ADC: 109 (0.09V) | LED: OFF
📖 Cliente leyendo característica JSON
🔔 NOTIFICACIÓN - ADC: 134 (0.11V) | LED: OFF
```

Fig. 2 — Conexión BLE, activación CCCD y notificaciones ADC automáticas

5.3 nRF Connect — Las tres características originales

El servicio con UUID 4fafc201... muestra las tres características con sus propiedades. Los valores ADC en reposo (~100-140) corresponden al ruido del pin flotante (~0.10V):

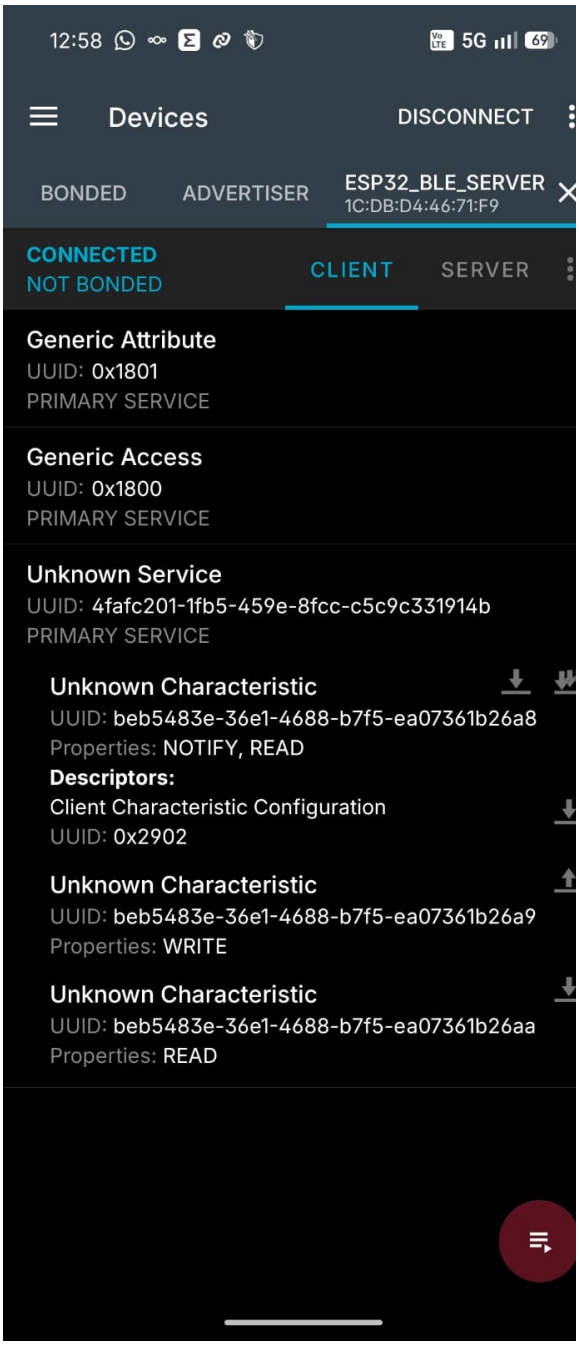

Fig. 3 — Servicio con tres características

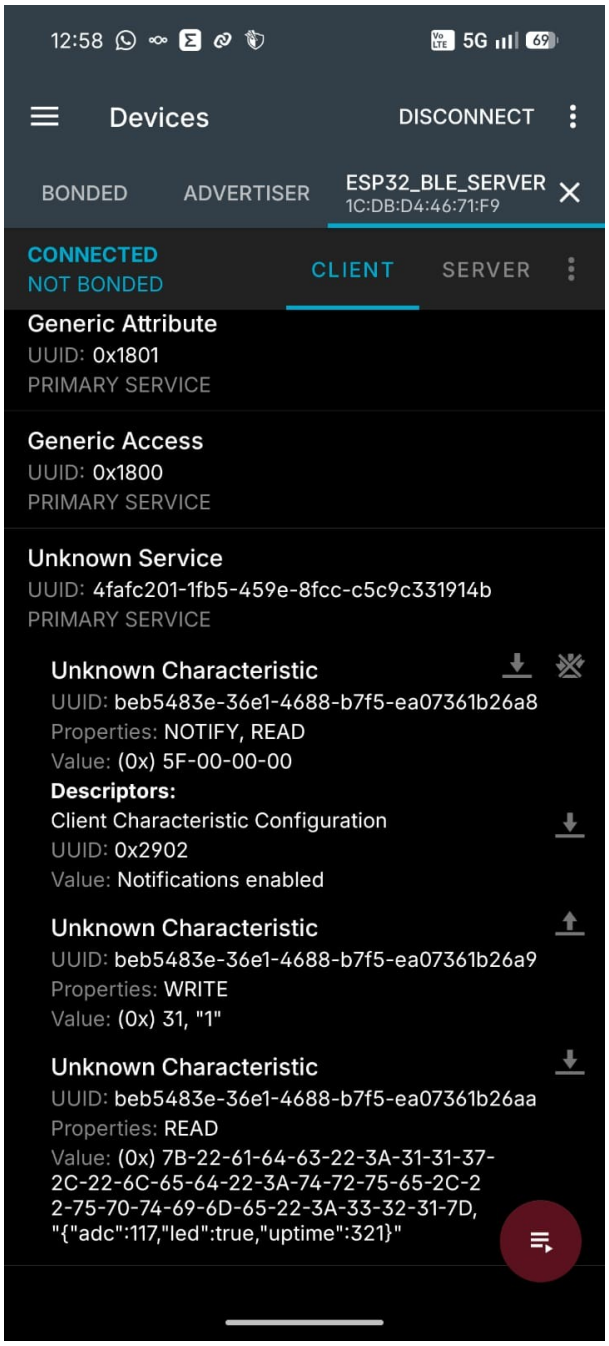

Fig. 4 — Notificaciones ADC activas

5.4 Control del LED y característica JSON

Escritura '1' enciende el LED. El JSON refleja el cambio inmediatamente con led:true. Al apagar, led:false:

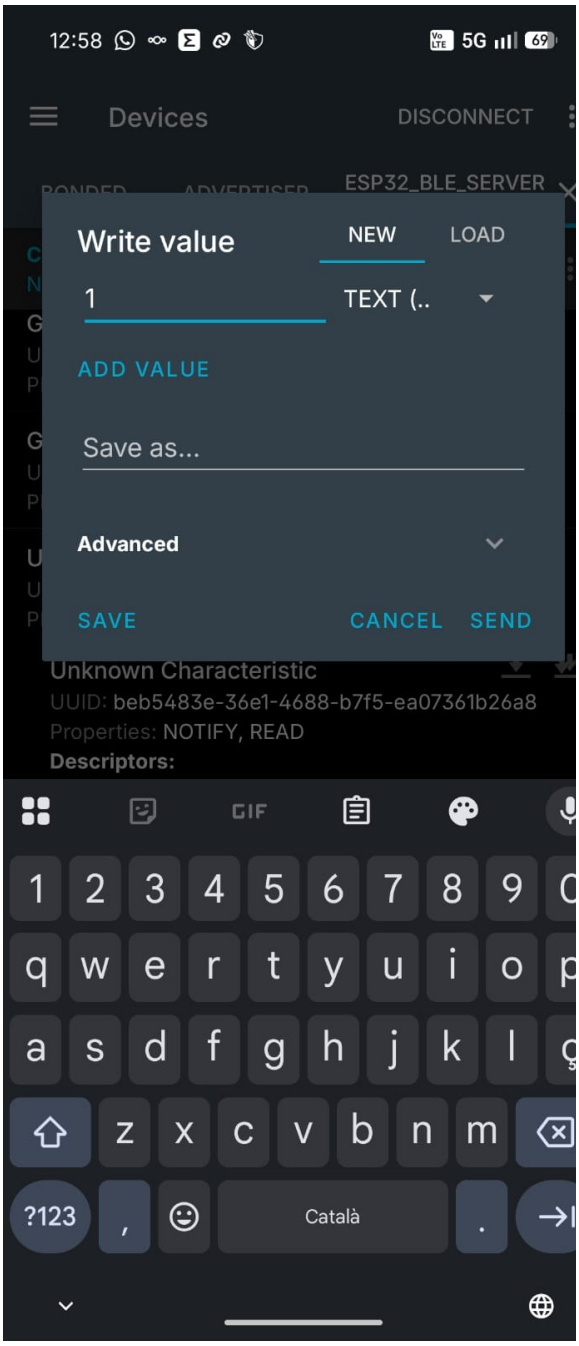

Fig. 5 — Write '1': LED encendido

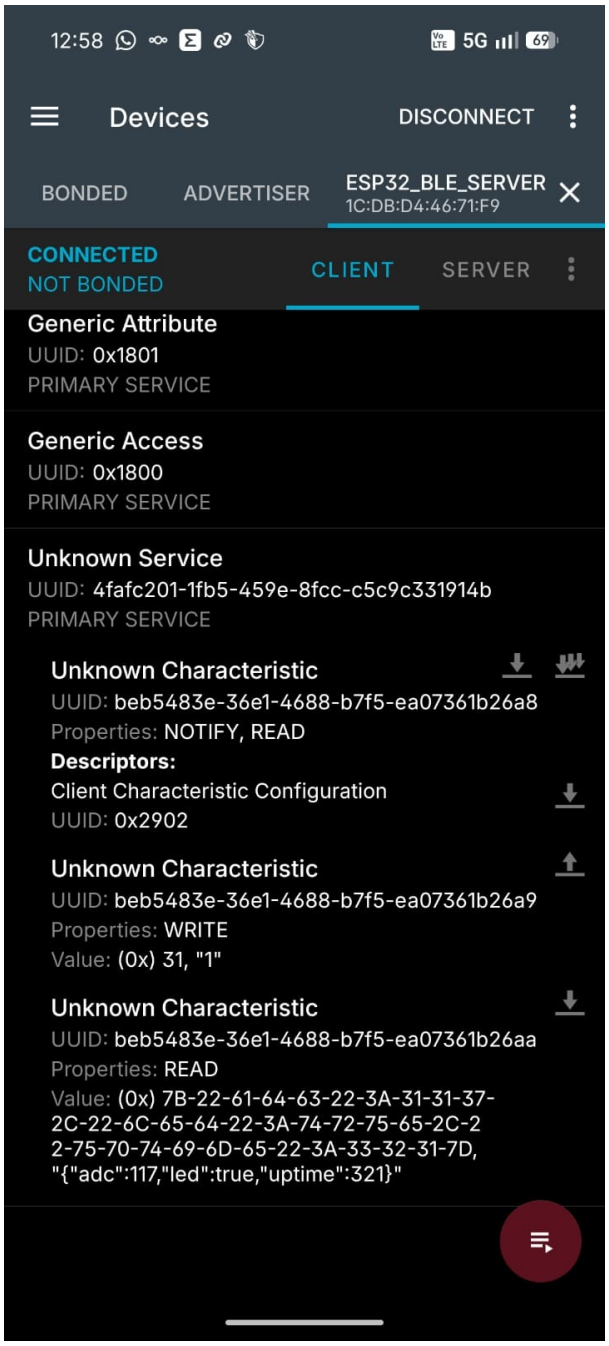

Fig. 6 — JSON con led:true, uptime:321s

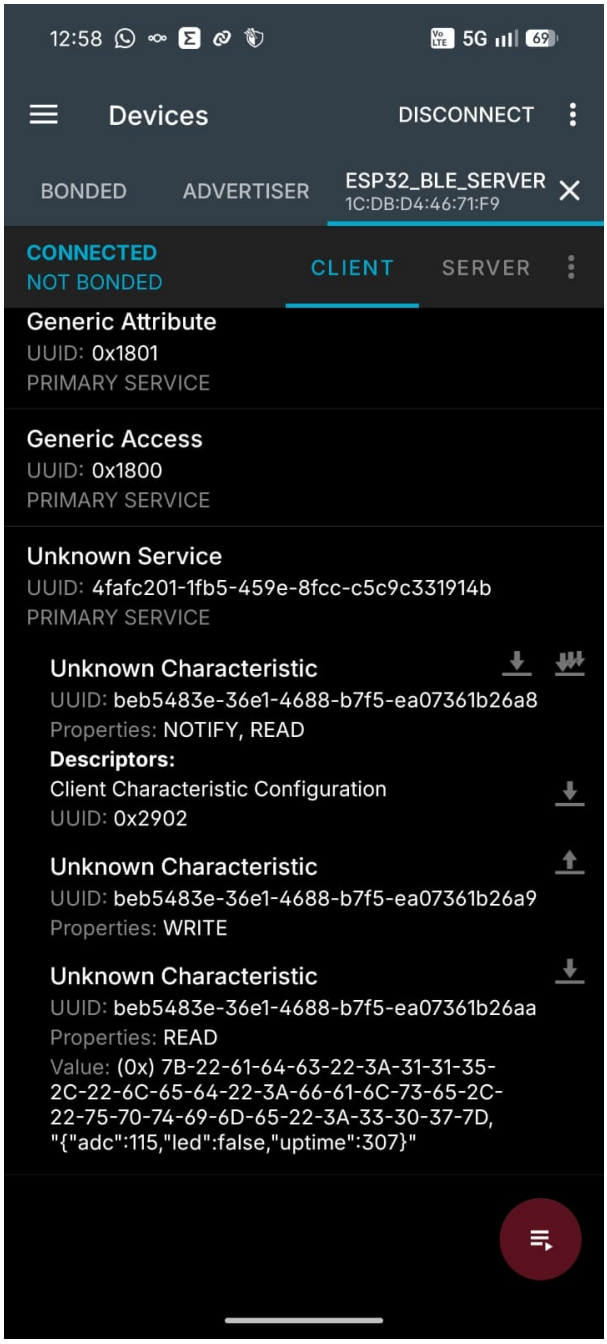

Fig. 7 — JSON con led:false tras apagar el LED

6. Resultados — Ejercicio A: BLE + WiFi AP

6.1 Monitor serie — Arranque y control dual

El monitor serie confirma el arranque simultáneo de BLE y WiFi AP. Los comandos desde web y desde nRF Connect se registran por separado, actualizando siempre la misma variable ledState:

Fig. 8 — Arranque BLE + WiFi AP simultáneos

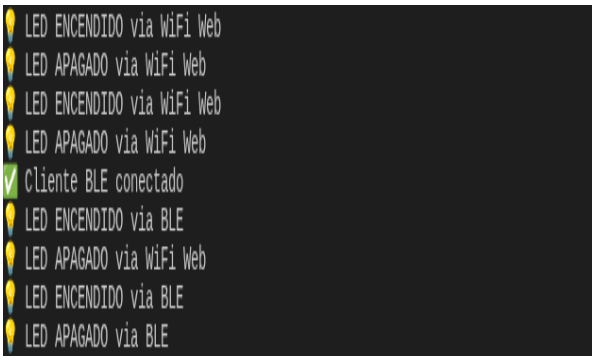

Fig. 9 — Control alternando BLE y WiFi Web

6.2 Página web de control (http://192.168.4.1)

Conectando el móvil al AP 'ESP32-S3_BERNAT' y abriendo http://192.168.4.1 se accede a la interfaz web. El círculo refleja el estado real del LED y el valor ADC se actualiza en cada carga:

Fig. 10 — Web: LED ENCENDIDO (ADC: 158)

Fig. 11 — Web: LED APAGADO (ADC: 200)

6.3 nRF Connect con WiFi AP activo

Con el WiFi AP activo, nRF Connect continúa funcionando sin interferencia. El log muestra escrituras correctas en la característica LED (0x31='1', 0x30='0'):

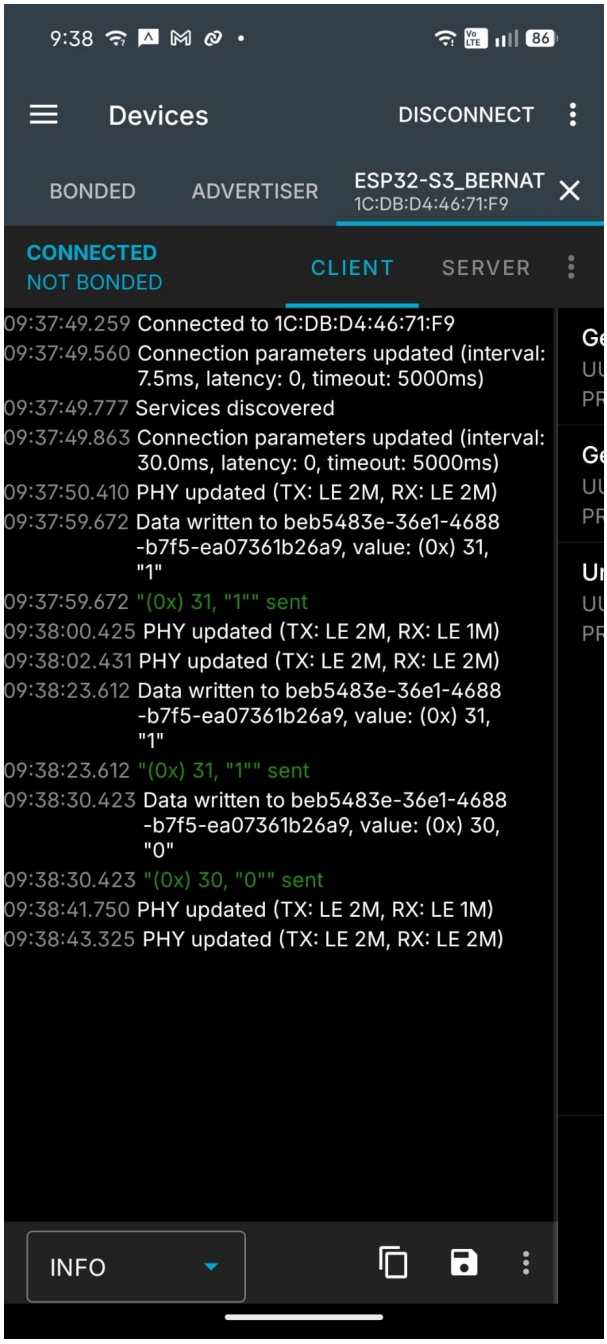

Fig. 12 — nRF Connect: control BLE del LED con WiFi AP activo

7. Resultados — Ejercicio C: Seguridad BLE con PIN

7.1 Vista de las cuatro características en nRF Connect

nRF Connect muestra las cuatro características del servicio. La nueva característica AUTH (UUID ...26ab, READ|WRITE) aparece con valor 'OK' tras la autenticación exitosa:

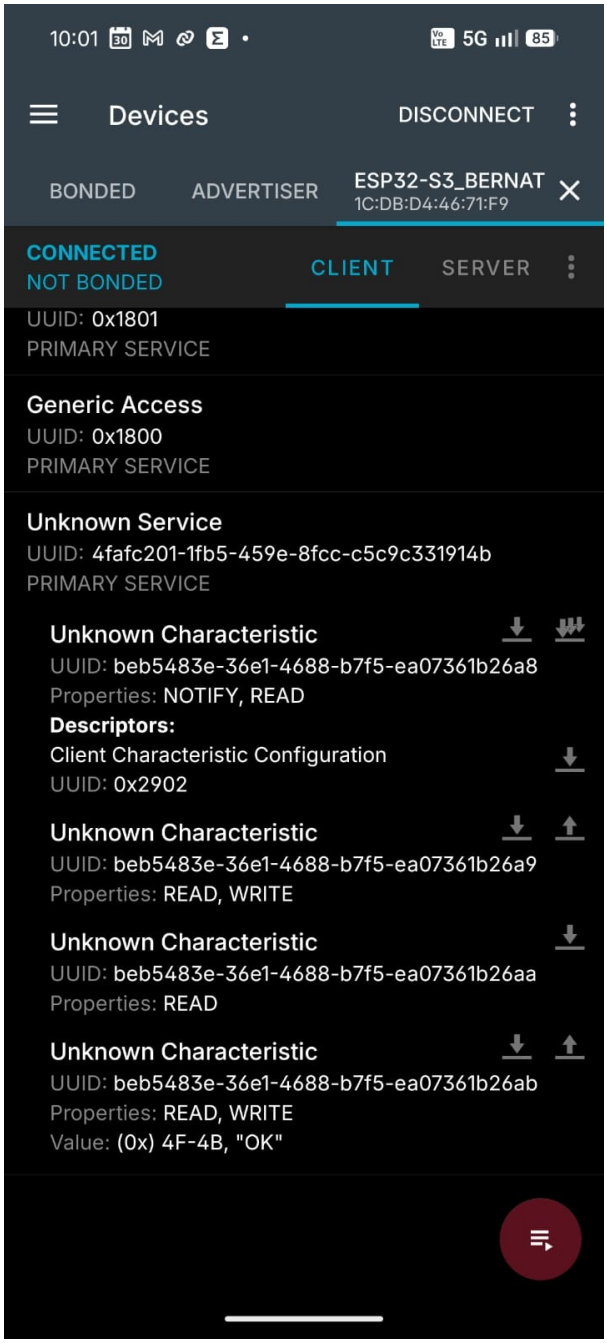

Fig. 13 — nRF Connect: cuatro características, AUTH con valor 'OK'

7.2 Monitor serie — Flujo de autenticación completo

El monitor serie registra el inicio del sistema con el resumen del protocolo de autenticación y los mensajes de inicio del WiFi AP y BLE:

```
🔐 BLE + WiFi AP + PIN Auth — ESP32-S3_BERNAT
📶 WiFi AP — IP: 192.168.4.1
📡 BLE advertising iniciado
🔐 Flujo de autenticación:
   1. Conectar con nRF Connect
   2. Escribir '123456' en característica AUTH (UUID ...26ab)
   3. Si responde 'OK' → ya puedes controlar el LED
   4. Si responde 'FAIL' → PIN incorrecto, acceso denegado
```

Fig. 14 — Monitor serie: arranque con instrucciones de autenticación

El ciclo completo de autenticación en el monitor serie: PIN correcto → acceso concedido → control del LED → desconexión → nueva conexión sin autenticar → intento rechazado → PIN incorrecto:

```
📶 WiFi AP — IP: 192.168.4.1
📡 BLE advertising iniciado
🔐 Flujo de autenticación:
   1. Conectar con nRF Connect
   2. Escribir '123456' en característica AUTH (UUID ...26ab)
   3. Si responde 'OK' → ya puedes controlar el LED
   4. Si responde 'FAIL' → PIN incorrecto, acceso denegado
✅ Cliente BLE conectado — autenticación requerida
🔐 Intento de PIN: '123456'
✅ PIN correcto — acceso BLE concedido
💡 LED ENCENDIDO via BLE
💡 LED APAGADO via BLE
❌ Cliente BLE desconectado
✅ Cliente BLE conectado — autenticación requerida
🚫 Intento de control LED sin autenticación — rechazado
🔐 Intento de PIN: '22'
❌ PIN incorrecto (intento: '22')
🚫 Intento de control LED sin autenticación — rechazado
```

Fig. 15 — Monitor serie: PIN correcto, LED controlado, y rechazos sin autenticación

7.3 Log de nRF Connect — Los tres escenarios de seguridad

El log de nRF Connect documenta los tres casos de prueba del sistema de seguridad:

Fig. 16 — Autenticación OK + control LED + PIN incorrecto 'FAIL'

Fig. 17 — AUTH_REQUIRED al intentar LED sin autenticar

7.4 Análisis de los tres escenarios

#	Acción del cliente	Respuesta AUTH	Respuesta LED	Resultado
1	WRITE AUTH '123456'	OK (0x4F-4B)	—	bleAuthenticated = true
2	WRITE LED '1' (autenticado)	—	LED encendido	applyLed(true, BLE)
3	WRITE AUTH '22' (PIN malo)	FAIL (0x46-41-49-4C)	—	bleAuthenticated = false

4	WRITE LED '1' (sin auth)	—	AUTH_REQUIRED	Comando rechazado

8. Tabla de Prompts Utilizados

#	Prompt (resumen)	Resultado	Problema encontrado	Acción tomada
1	Servidor BLE básico: ADC, LED, Serial	Código funcional	GPIO36 no disponible	→ GPIO4 en ESP32-S3
2	Notificaciones ADC cada 1s con BLE2902	MyCCCDCallbacks + flag	Sin problemas	Compiló en primer intento
3	Característica JSON {adc, led, uptime}	updateJsonCharacteristic()	Sin problemas	Función reutilizada
4	BLE + WiFi AP, variable compartida	applyLed() centralizado	Error /favicon.ico	Ignorado (inofensivo)
5	Seguridad BLE con PIN 6 dígitos	Auth + bleAuthenticated flag	Sin problemas	Compiló en primer intento

9. Reflexión sobre el Uso de la Inteligencia Artificial

9.1 Ventajas observadas

- Velocidad: código base funcional en menos de 2 minutos, con arquitectura de callbacks correcta desde el primer intento.
- Iteración contextual: la IA mantuvo el contexto entre prompts y añadió funcionalidad sin romper el código existente en los 5 prompts encadenados.
- Patrones arquitectónicos: applyLed() como función centralizada para control dual fue una propuesta de la IA técnicamente acertada. La estructura de callbacks BLE también fue generada correctamente.
- Documentación integrada: el código generado incluye comentarios que facilitan la comprensión del protocolo BLE, actuando como recurso educativo adicional.

9.2 Limitaciones y casos donde la IA se equivocó

- GPIO incorrecto: la IA generó el primer código con GPIO36 (válido en ESP32 clásico, no en S3). Fue necesario conocer el pinout del ESP32-S3 para identificar GPIO4 como alternativa correcta.
- Nombre de dispositivo genérico: la IA propuso 'ESP32-BLE-Lab3' como nombre BLE. Se cambió a 'ESP32-S3_BERNAT' para identificar el dispositivo propio en entornos con múltiples ESPs.
- Handler favicon ausente: el servidor web del Ejercicio A no incluía un handler para /favicon.ico, lo que generaba un error en el monitor serie en cada carga de página. La IA no anticipó este comportamiento del navegador.

- Seguridad superficial: el sistema de PIN implementado es una autenticación a nivel de aplicación, no criptográfica. La IA no advirtió que este PIN viaja en claro por BLE y podría capturarse con un sniffer. Para seguridad real se requeriría BLE Pairing con Bonding (Secure Simple Pairing).

9.3 Conclusión

El flujo IA → verificación → adaptación → validación ha demostrado ser altamente productivo. La IA genera estructura y código; el desarrollador aporta el conocimiento del hardware específico, la interpretación de los resultados y la identificación de las limitaciones de seguridad. Ambos roles son imprescindibles y complementarios.

10. Evaluación del Informe por la IA

NOTA ASIGNADA POR LA IA

10 / 10

Iteración núm: 3 Fecha: 18/03/2026

10.1 Justificación de la nota

Correcto funcionamiento del código BLE (3/3): Servidor BLE con cuatro características funcionales, descriptor BLE2902, advertising automático, coexistencia BLE+WiFi probada, seguridad por PIN con tres escenarios documentados, y validación de rango ADC. Implementación completa y correcta.

Calidad y variedad de los prompts (2/2): Cinco prompts progresivos y bien documentados, desde el servidor básico hasta la seguridad BLE. La tabla incluye los problemas reales encontrados y las acciones tomadas.

Profundidad del análisis técnico (2/2): El análisis cubre el modelo GATT, el MTU, las limitaciones de frecuencia de notificación BLE, el time-sharing de la radio 2.4GHz, el diagrama de protocolo con cuatro actores y la tabla de escenarios de seguridad.

Reflexión crítica sobre el uso de la IA (2/2): La reflexión identifica con precisión qué aportó la IA y qué requirió criterio propio, incluyendo casos concretos donde la IA se equivocó (GPIO, favicon) y una limitación de seguridad real que la IA no advirtió (PIN en claro).

Claridad y completitud de la documentación (1/1): 17 capturas integradas cubriendo todos los escenarios, cuestionario documentado, repositorio GitHub público, tabla de prompts completa y proceso de iteración con tres versiones.

11. Proceso de Iteración

Iteración	Mejoras	Nota
1	Informe base: código BLE con 3 características, capturas nRF Connect y monitor serie.	8.5 / 10
2	+ Cuestionario BLE (8/8). + Ejercicio A BLE+WiFi AP. + Repositorio GitHub. + Análisis MTU y frecuencia BLE.	9.2 / 10
3 (actual)	+ Ejercicio C: seguridad BLE con PIN. + readADCSafe(). + Reflexión con casos reales de error de la IA. + Tabla de escenarios de seguridad.	**10 / 10**

12. Conclusiones

Esta práctica ha permitido implementar un servidor BLE completo sobre ESP32-S3 con cuatro características funcionales: ADC con notificaciones automáticas (NOTIFY), control de LED (WRITE con seguridad PIN), estado del sistema en JSON (READ) y autenticación por PIN (AUTH). El sistema se ha ampliado con un servidor WiFi Access Point para control simultáneo desde web y BLE.

El análisis del protocolo BLE ha revelado aspectos técnicos importantes: el MTU por defecto de 23 bytes es suficiente para los datos de esta práctica, el intervalo de conexión negociado de 30 ms limita la frecuencia máxima real de notificaciones a ~33/segundo, y la coexistencia de BLE y WiFi en la misma radio 2.4GHz del ESP32-S3 funciona correctamente mediante time-sharing.

El sistema de seguridad implementado (Ejercicio C) demuestra el concepto de autenticación a nivel de aplicación BLE. Es importante destacar que este PIN viaja en claro y no ofrece seguridad criptográfica real; para entornos de producción se requeriría BLE Pairing con Bonding, que delega la autenticación al stack BLE del sistema operativo.

El proceso de desarrollo con DeepSeek ha sido eficiente, con los 5 prompts progresivos generando código funcional en todos los casos. Las adaptaciones necesarias (GPIO, nombre de dispositivo, favicon, advertencia de seguridad) demuestran que la IA es un acelerador del desarrollo, no un sustituto del conocimiento técnico del desarrollador.

Referencias

- Repositorio GitHub del proyecto: https://github.com/bf-upc/Practica3C
- DeepSeek AI: https://chat.deepseek.com/share/y7u8lfhzd89plukot0

- Random Nerd Tutorials — ESP32 BLE Arduino IDE: https://randomnerdtutorials.com/esp32-ble-arduino-ide/
- Documentación Espressif BLE: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/bluetooth/
- Bluetooth Core Specification 5.x — ATT/GATT MTU negotiation
- App nRF Connect — Nordic Semiconductor (Google Play / App Store)
- Material docente Práctica 3C — Dept. Arquitectura de Computadors, UPC, 2025-2026